

03__baseline__models

June 8, 2026

1 03 — Modelos de Referência (Baseline)

TCC PUC — Ciência de Dados e Big Data Evandro Ribeiro Gomes Coelho

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import TimeSeriesSplit
import warnings
warnings.filterwarnings('ignore')

plt.rcParams['figure.figsize'] = (14, 5)
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3

PROC = '../data/processed/'

def compute_metrics(y_true, y_pred, model_name):
    """Calcula MAE, RMSE, R² e MAPE."""
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    # MAPE com proteção contra zeros
    mask = y_true != 0
    mape = np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100
    return {'Modelo': model_name, 'MAE': round(mae,4), 'RMSE': round(rmse,4),
            'R²': round(r2,5), 'MAPE (%)': round(mape,3)}

print('OK')
```

OK

```
[2]: # Carrega dados de features
df = pd.read_csv(PROC + 'df_features.csv', parse_dates=['Timestamp'])
df = df.sort_values('Timestamp').reset_index(drop=True)
```

```

# Split temporal: 75% treino / 25% teste (mesma proporção do TCC2)
split_idx = int(len(df) * 0.75)
df_train = df.iloc[:split_idx].copy()
df_test = df.iloc[split_idx:].copy()

y_train = df_train['Value'].values
y_test = df_test['Value'].values

print(f'Total : {len(df)} registros')
print(f'Treino : {len(df_train)} registros ({df_train["Timestamp"].min()} → {df_train["Timestamp"].max()})')
print(f'Teste : {len(df_test)} registros ({df_test["Timestamp"].min()} → {df_test["Timestamp"].max()})')

```

```

Total : 16632 registros
Treino : 12474 registros (2015-07-04 14:30:00+00:00 → 2017-04-11 20:30:00+00:00)
Teste : 4158 registros (2017-04-11 21:30:00+00:00 → 2017-11-08 13:30:00+00:00)

```

1.1 1. Média Móvel Simples (MMS)

Revisado no TCC1 — prevê o próximo valor como média dos N períodos anteriores.

```

[3]: results = []

for window in [6, 12, 24]:
    # Usa toda a série para calcular rolling, aplica no conjunto de teste
    series_full = df['Value'].values
    pred_mms = np.full(len(series_full), np.nan)
    for i in range(window, len(series_full)):
        pred_mms[i] = series_full[i-window:i].mean()

    y_pred_test = pred_mms[split_idx:]
    # Remove NaN no inicio
    mask = ~np.isnan(y_pred_test)
    metrics = compute_metrics(y_test[mask], y_pred_test[mask], f'MMS_{window}')
    results.append(metrics)
    print(f"MMS w={window:2d}: MAE={metrics['MAE']:.4f} RMSE={metrics['RMSE']:.4f} R²={metrics['R²']:.5f}")

# Melhor MMS
best_mms_w = 24
series_full = df['Value'].values
pred_mms_best = pd.Series(series_full).rolling(best_mms_w).mean().shift(1).values
y_pred_mms = pred_mms_best[split_idx:]

```

MMS w= 6: MAE=1477.4809 RMSE=2026.9756 $R^2=0.46479$
MMS w=12: MAE=1492.1786 RMSE=2020.4274 $R^2=0.46825$
MMS w=24: MAE=1639.5833 RMSE=2150.1403 $R^2=0.39778$

1.2 2. Média Móvel Exponencial (MME)

Revisado no TCC1 — pesos exponencialmente decrescentes para dados mais antigos.

```
[4]: for alpha in [0.1, 0.3, 0.5]:
    ema = pd.Series(df['Value']).ewm(alpha=alpha, adjust=False).mean().shift(1).
    ↪values
    y_pred_ema = ema[split_idx:]
    mask = ~np.isnan(y_pred_ema)
    metrics = compute_metrics(y_test[mask], y_pred_ema[mask], f'MME_
    ↪(={alpha})')
    results.append(metrics)
    print(f"MME ={alpha}: MAE={metrics['MAE']:.4f} RMSE={metrics['RMSE']:.4f}
    ↪ R²={metrics['R²']:.5f}")

# Melhor para plot
best_alpha = 0.3
y_pred_mme = pd.Series(df['Value']).ewm(alpha=best_alpha, adjust=False).mean().
    ↪shift(1).values[split_idx:]
```

MME =0.1: MAE=1519.6861 RMSE=1996.9895 $R^2=0.48051$
MME =0.3: MAE=1296.0484 RMSE=1802.9547 $R^2=0.57656$
MME =0.5: MAE=1197.6700 RMSE=1709.2959 $R^2=0.61941$

1.3 3. Regressão Linear

```
[5]: FEATURE_COLS = [
    'Temperature', 'temp_lag1', 'energy_lag1',
    'hr_sin', 'hr_cos', 'dow_sin', 'dow_cos',
    'is_weekend', 'is_holiday'
]

X_train = df_train[FEATURE_COLS].values
X_test = df_test[FEATURE_COLS].values

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

metrics_lr = compute_metrics(y_test, y_pred_lr, 'Regressão Linear')
results.append(metrics_lr)
print(f"Regressão Linear: MAE={metrics_lr['MAE']:.4f} RMSE={metrics_lr['RMSE']:.
    ↪.4f} R²={metrics_lr['R²']:.5f}")
```

```
# Coeficientes
coef_df = pd.DataFrame({'Feature': FEATURE_COLS, 'Coeficiente': lr.coef_})
coef_df = coef_df.sort_values('Coeficiente', key=abs, ascending=False)
print('\nCoeficientes da Regressão Linear:')
print(coef_df.to_string(index=False))
```

Regressão Linear: MAE=1083.8078 RMSE=1499.3586 $R^2=0.70716$

Coeficientes da Regressão Linear:

Feature	Coeficiente
is_weekend	-424.247557
is_holiday	-284.083425
dow_cos	161.674709
dow_sin	140.321840
hr_sin	121.322236
hr_cos	-92.705612
Temperature	-41.161355
temp_lag1	-2.438028
energy_lag1	0.834021

1.4 4. SARIMA

```
[6]: import pmdarima as pm
```

```
[7]: # Amostra menor para velocidade (últimas 2 semanas de treino + 2 dias de teste)
N_TRAIN_SARIMA = 24 * 14 # 2 semanas
N_TEST_SARIMA = 24 * 2 # 2 dias

y_train_s = y_train[-N_TRAIN_SARIMA:]
y_test_s = y_test[:N_TEST_SARIMA]

sarima_model = pm.auto_arima(
    y_train_s,
    seasonal=True,
    m=24, # sazonalidade diária
    stepwise=True,
    suppress_warnings=True,
    error_action='ignore',
    max_p=3, max_q=3,
    max_P=1, max_Q=1,
    information_criterion='aic',
)
y_pred_sarima = sarima_model.predict(n_periods=N_TEST_SARIMA)
print(f'SARIMA order: {sarima_model.order} | seasonal order: {sarima_model.
    ↪seasonal_order}')
```

```
metrics_sarima = compute_metrics(y_test_s, y_pred_sarima, 'SARIMA')
results.append(metrics_sarima)
print(f"SARIMA: MAE={metrics_sarima['MAE']:.4f} RMSE={metrics_sarima['RMSE']:.4f} R²={metrics_sarima['R²']:.5f}")
```

SARIMA order: (3, 0, 0) | seasonal order: (1, 0, 0, 24)
SARIMA: MAE=1449.6561 RMSE=1761.8203 R²=0.36011

1.5 5. Comparação dos Modelos Baseline

```
[8]: results_df = pd.DataFrame(results)
results_df = results_df.sort_values('R²', ascending=False).
      ↪reset_index(drop=True)
print('Resultados dos Modelos Baseline:')
display(results_df)
```

Resultados dos Modelos Baseline:

	Modelo	MAE	RMSE	R²	MAPE (%)
0	Regressão Linear	1083.8078	1499.3586	0.70716	27.487
1	MME (=0.5)	1197.6700	1709.2959	0.61941	29.263
2	MME (=0.3)	1296.0484	1802.9547	0.57656	33.667
3	MME (=0.1)	1519.6861	1996.9895	0.48051	45.349
4	MMS (w=12)	1492.1786	2020.4274	0.46825	41.823
5	MMS (w=6)	1477.4809	2026.9756	0.46479	38.345
6	MMS (w=24)	1639.5833	2150.1403	0.39778	50.720
7	SARIMA	1449.6561	1761.8203	0.36011	29.911

```
[9]: # Gráfico de previsão (sample dos primeiros 7 dias de teste)
N_SHOW = 24 * 7
t_show = df_test['Timestamp'].values[:N_SHOW]
y_show = y_test[:N_SHOW]

fig, axes = plt.subplots(2, 1, figsize=(16, 10))

for ax in axes:
    ax.plot(t_show, y_show, label='Real', color='black', linewidth=1.2,
            ↪zorder=5)

# Subplot 1: MMS e MME
mask_mms = ~np.isnan(y_pred_mms[:N_SHOW])
axes[0].plot(t_show[mask_mms], y_pred_mms[:N_SHOW][mask_mms], label=f'MMS_
            ↪(w={best_mms_w})', linestyle='--')
mask_mme = ~np.isnan(y_pred_mme[:N_SHOW])
axes[0].plot(t_show[mask_mme], y_pred_mme[:N_SHOW][mask_mme], label=f'MME_
            ↪(={best_alpha})', linestyle='-')
axes[0].set_title('MMS vs MME - Primeiros 7 dias do conjunto de teste')
```

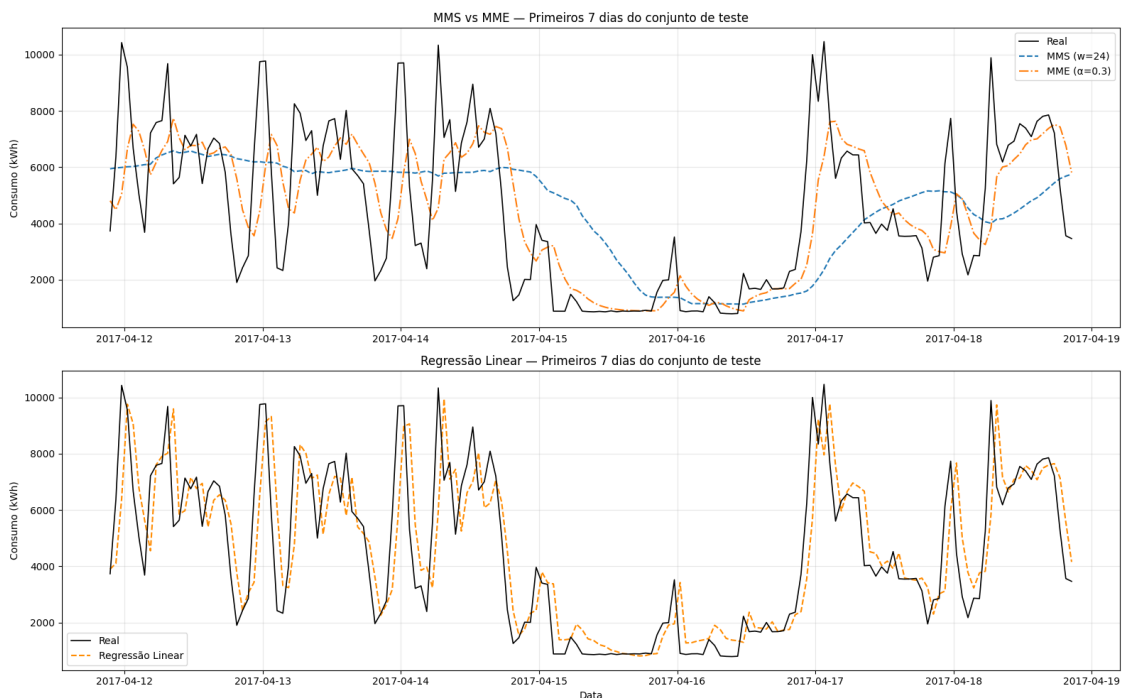
```

axes[0].set_ylabel('Consumo (kWh)')
axes[0].legend()

# Subplot 2: Regressão Linear
axes[1].plot(t_show, y_pred_lr[:N_SHOW], label='Regressão Linear',
             linestyle='--', color='darkorange')
axes[1].set_title('Regressão Linear - Primeiros 7 dias do conjunto de teste')
axes[1].set_ylabel('Consumo (kWh)')
axes[1].set_xlabel('Data')
axes[1].legend()

plt.tight_layout()
plt.show()

```



```

[10]: # Salva resultados baseline para notebook 05
results_df.to_csv(PROC + 'results_baseline.csv', index=False)
print(f'Resultados salvos: {PROC}results_baseline.csv')

```

Resultados salvos: ../data/processed/results_baseline.csv